

CALORIE & NUTRITION TRACKER



CALORIE & NUTRITION TRACKER

Features Required:

1. Log daily food intake with meal categories (Breakfast, Lunch, Dinner, Snacks)
2. Display nutritional information: Calories, Protein, Carbs, Fats per food item
3. Calculate and display total daily nutrition summary
4. Track nutrition across multiple days using file storage
5. Provide daily calorie goal comparison and progress display
6. Allow searching food items from a predefined nutrition database

Concepts to be Used: Classes, Objects, Vectors, Loops, Conditional Statements, File Handling, String Functions, Standard Input / Output Streams

Technology Used: C++ Programming Language

Output: Console-based Calorie & Nutrition Tracker. Daily nutrition summary displayed after logging meals.

1. Introduction

The Calorie & Nutrition Tracker is a console-based application developed using the C++ programming language. The main objective of this project is to help users monitor their daily food intake and nutritional consumption across categories such as Calories, Protein, Carbohydrates, and Fats. The program is menu-driven, allowing the user to log meals, view nutritional summaries, and compare intake against daily goals. Each food item is represented as an object of a class containing its nutritional values. The user logs meals by selecting food items, and the system calculates and displays the cumulative nutrition. This project demonstrates the use of classes, objects, vectors, file streams, and standard input/output in C++ programming.

2. Objectives

- To develop a menu-driven nutrition tracking application using C++
- To apply Object-Oriented Programming concepts such as classes and objects
- To maintain a predefined food database using vectors
- To implement meal logging across Breakfast, Lunch, Dinner, and Snacks
- To calculate total daily calorie and macro-nutrient intake
- To compare daily intake against a user-defined calorie goal
- To store and retrieve food logs using file streams

- To improve logical thinking and OOP-based programming skills

3. System Features

3.1 Food Database The system maintains a built-in database of common food items stored as a vector of Food objects. Each object holds the item's name, calories, protein, carbohydrates, and fat values per serving. The user selects items from this database when logging meals.

3.2 Meal Logging The program displays a menu with four meal categories: Breakfast, Lunch, Dinner, and Snacks. The user selects a category, then chooses food items to add. Multiple items can be added per meal using a loop. The meal data is saved to a file using ofstream for persistence.

3.3 Nutrition Calculation The user's selections are accumulated across all meals using member functions of the Tracker class. Calories, protein, carbs, and fats are summed and displayed as a daily total. If the total exceeds the user's daily calorie goal, a warning message is displayed.

3.4 Daily Summary & Goal Tracking After logging meals, the system displays a complete nutritional breakdown using a display function. The summary shows total intake, remaining calories, and a comparison against the user's set goal.

4. Technology & Concepts Used

4.1 C++ Programming Concepts

- Classes & Objects – Food class to store item data; Tracker class to manage logging and totals
- Vectors – to store the food database and logged meal entries
- Switch-case – to select meal categories and menu options
- If-else statements – to validate input and compare against goal
- File Streams – ofstream and ifstream to save and load daily logs
- cin / cout – for standard input and output

4.2 Program Control Technique The program uses a menu-driven approach where the user's selection controls the execution flow. Each meal category is handled as a separate case inside the switch statement. A do-while loop keeps the main menu active until the user chooses to exit. Member functions of the Tracker class handle all core logic.

5. System Design

5.1 Data Used

- class Food { string name; int calories; float protein, carbs, fats; } – food item class
- class Tracker { vector<Food> log; int goal; void addMeal(); void showSummary(); } – tracker class

- int choice – stores selected menu option
- int totalCalories – accumulates daily calorie count
- float totalProtein, totalCarbs, totalFats – accumulate macronutrient totals
- int goal – stores user-defined daily calorie goal
- ofstream / ifstream – file stream objects for log storage

These variables and objects help in controlling the tracker flow and nutrition calculation.

6. Workflow

Main Program Flow:

1. Display main menu (Log Meal / View Summary / Set Goal / Exit)
2. Accept user category choice
3. Display food database for selected meal category
4. Accept user food selections using a loop
5. Calculate nutrition values and update Tracker object
6. Save logged data to file using ofstream
7. Display daily nutrition summary with goal comparison

7. Sample Output (Console)

==== CALORIE & NUTRITION TRACKER =====

1. Log Meal
2. View Daily Summary
3. Set Calorie Goal
4. Exit

Enter your choice: 1

Select Meal: 1.Breakfast 2.Lunch 3.Dinner 4.Snacks

Your choice: 2

Food Database:

1. Rice (1 cup) - 206 kcal | P:4g C:45g F:0.4g
2. Chicken (100g) - 239 kcal | P:27g C:0g F:14g

Select item (0 to stop): 2

--- Daily Summary ---

Total Calories : 860 / 2000 kcal

Protein : 54 g

Carbohydrates : 90 g

Fats : 28 g

Remaining : 1140 kcal

Note: The displayed values are examples. Actual values depend on the food items logged by the user.

8. Conclusion

The Calorie & Nutrition Tracker is a simple and effective console-based application developed using C++ programming. It demonstrates the use of Object-Oriented Programming, menu-driven design, file streams, and vectors. This project helps beginners understand class design, data encapsulation, and program control flow. It can be further enhanced by adding inheritance for meal types, operator overloading for nutrition addition, weekly reports, BMI calculation, exercise calorie burn tracking, and a larger food database with search functionality.

Sample code

```
#include <iostream>
#include <string>
using namespace std;
class Food {
public:
    string foodName;
    int calories;
    int protein;
    int carbs;
    int fats;
    void addFood() {
        cin.ignore();
```

```

        cout << "\nEnter Food Name: ";
        getline(cin, foodName);
        cout << "Enter Calories: ";
        cin >> calories;
        cout << "Enter Protein (g): ";
        cin >> protein;
        cout << "Enter Carbohydrates (g): ";
        cin >> carbs;
        cout << "Enter Fats (g): ";
        cin >> fats;
    }

    void displayFood() {
        cout << "\n-----";
        cout << "\nFood Name    : " << foodName;
        cout << "\nCalories    : " << calories;
        cout << "\nProtein     : " << protein << " g";
        cout << "\nCarbohydrates : " << carbs << " g";
        cout << "\nFats        : " << fats << " g";
        cout << "\n-----";
    }
};

int main() {
    Food foods[100];
    int count = 0;
    int choice;
    int totalCalories = 0;
    int totalProtein = 0;
    int totalCarbs = 0;
    int totalFats = 0;
    do {
        cout << "\n\n===== CALORIE AND NUTRITION TRACKER =====";
        cout << "\n1. Add Food";
    }

```

```

cout << "\n2. View All Foods";
cout << "\n3. View Total Nutrition";
cout << "\n4. Exit";
cout << "\nEnter your choice: ";
cin >> choice;
switch(choice) {
case 1:
    foods[count].addFood()
    totalCalories += foods[count].calories;
    totalProtein += foods[count].protein;
    totalCarbs += foods[count].carbs;
    totalFats += foods[count].fats;
    count++;
    cout << "\nFood Added Successfully!";
    break;
case 2:
    if(count == 0) {
        cout << "\nNo Food Records Available!";
    }
    else {
        for(int i = 0; i < count; i++) {
            foods[i].displayFood();
        }
    }
    break;
case 3:
    cout << "\n===== TOTAL NUTRITION =====";
    cout << "\nTotal Calories      : "
        << totalCalories;
    cout << "\nTotal Protein      : "
        << totalProtein << " g";
    cout << "\nTotal Carbohydrates : "

```

```

        << totalCarbs << " g";
    cout << "\nTotal Fats      : "
        << totalFats << " g";
    cout << "\n=====";
    break;
case 4:
    cout << "\nThank You for Using the Tracker!";
    break;
default:
    cout << "\nInvalid Choice!";
}
} while(choice != 4);
return 0;
}

```